

Commit 3cad777

Browse files

ActuallyNull committed last month

Model 2.0, Spreadsheet updated: improved ecg data augmentation, implemented ecg data generator for on the fly data augmentation, improved data normalisation, adjusted parameters in data augmentation, not much done in model

main

1 parent f3a27a3 commit 3cad777

Filter files...

- data_augmentor.py
- data_generator.py
- data_processing.py
- model.py

4 files changed +91 -36 lines changed

Search within code

data_augmentor.py

@@ -2,23 +2,26 @@
from scipy.signal import resample

class ECG_Augmentor:
- def __init__(self, fs=360):
+ def __init__(self, fs=360, strong=False):
self.fs = fs # sampling frequency
self.strong = strong

def add_gaussian_noise(self, signal, noise_level=0.01):
noise = np.random.normal(0, noise_level * np.max(signal), size=signal.shape)
max_val = np.max(np.abs(signal))
scale = max(noise_level * max_val, 1e-6)
noise = np.random.normal(0, scale, size=signal.shape)
return signal + noise

def baseline_wander(self, signal, freq=0.3, amp=0.05):
t = np.arange(len(signal)) / self.fs
drift = amp * np.sin(2 * np.pi * freq * t)
return signal + drift

- def time_shift(self, signal, shift_max=0.1):
+ def time_shift(self, signal, shift_max=0.05): # Reduce shift to 5%
shift = int(np.random.uniform(-shift_max, shift_max) * len(signal))
return np.roll(signal, shift)

- def time_scale(self, signal, scale_range=(0.9, 1.1)):
+ def time_scale(self, signal, scale_range=(0.95, 1.05)): # Reduce scaling
scale = np.random.uniform(*scale_range)
new_len = int(len(signal) * scale)
scaled = resample(signal, new_len)
@@ -28,14 +31,25 @@ def time_scale(self, signal, scale_range=(0.9, 1.1)):
else:
return np.pad(scaled, (0, len(signal) - len(scaled)), mode='constant')

- def amplitude_scale(self, signal, scale_range=(0.9, 1.1)):
+ def amplitude_scale(self, signal, scale_range=(0.95, 1.05)):
scale = np.random.uniform(*scale_range)
return signal * scale

- def amplitude_shift(self, signal, shift_range=(-0.1, 0.1)):
+ def amplitude_shift(self, signal, shift_range=(-0.05, 0.05)):
shift = np.random.uniform(*shift_range) * np.max(signal)
return signal + shift

def random_crop(self, signal, crop_size=0.95):
Randomly crop a segment and pad to original length
orig_len = len(signal)
crop_len = int(orig_len * crop_size)
start = np.random.randint(0, orig_len - crop_len)
cropped = signal[start:start+crop_len]
return np.pad(cropped, (0, orig_len - crop_len), mode='constant')

def invert(self, signal):
return -signal

def mixup(self, sig1, sig2, alpha=0.2):
lam = np.random.beta(alpha, alpha)
return lam * sig1 + (1 - lam) * sig2, lam
@@ -44,19 +58,28 @@ def augment(self, signal, mix_signal=None):
"""Apply a random set of augmentations"""
aug_signal = signal.copy()

Standard augmentations with reduced probabilities
if np.random.rand() < 0.5:
aug_signal = self.add_gaussian_noise(aug_signal)
- if np.random.rand() < 0.3:
+ if np.random.rand() < 0.2:
aug_signal = self.baseline_wander(aug_signal)
- if np.random.rand() < 0.5:
+ if np.random.rand() < 0.1:
aug_signal = self.time_shift(aug_signal)
- if np.random.rand() < 0.4:
+ if np.random.rand() < 0.1:
aug_signal = self.time_scale(aug_signal)
- if np.random.rand() < 0.5:
+ if np.random.rand() < 0.2:
aug_signal = self.amplitude_scale(aug_signal)
- if np.random.rand() < 0.3:
+ if np.random.rand() < 0.1:
aug_signal = self.amplitude_shift(aug_signal)
- if mix_signal is not None and np.random.rand() < 0.3:
+ aug_signal, _ = self.mixup(aug_signal, mix_signal)

disabled for now
if mix_signal is not None and np.random.rand() < 0.1:
aug_signal, _ = self.mixup(aug_signal, mix_signal)

Stronger augmentations if enabled
if self.strong:
if np.random.rand() < 0.1:
aug_signal = self.random_crop(aug_signal)
if np.random.rand() < 0.05:
aug_signal = self.invert(aug_signal)

return aug_signal

data_generator.py

@@ -1,19 +1,17 @@
import numpy as np
- import tensorflow as tf
from keras.utils import Sequence
- from data_processing import create_train_val_test_splits
- from data_augmentor import ECG_Augmentor

Custom Data Generator
class ECGDataGenerator(Sequence):
- def __init__(self, X, y, batch_size=64, augmentor=None, shuffle=True):
+ def __init__(self, X, y, batch_size=64, augmentor=None, shuffle=True, signal_length=3000):
self.X = X
self.y = y
self.batch_size = batch_size
self.augmentor = augmentor
self.shuffle = shuffle
self.signal_length = signal_length
self.indices = np.arange(len(self.X))
self.on_epoch_end()

@@ -25,11 +23,28 @@ def __getitem__(self, index):
batch_indices = self.indices[index * self.batch_size:(index + 1) * self.batch_size]
X_batch = self.X[batch_indices].copy()
y_batch = self.y[batch_indices]

-
Apply augmentation if augmentor is provided
if self.augmentor is not None:
X_batch = np.array([self.augmentor.augment(x.squeeze()) for x in X_batch])

X_aug = []
for x in X_batch:
aug = self.augmentor.augment(x.squeeze())
Ensure output is always (signal_length,)
if len(aug) > self.signal_length:
aug = aug[:self.signal_length]
elif len(aug) < self.signal_length:
aug = np.pad(aug, (0, self.signal_length - len(aug)))
Normalise after augmentation
aug = (aug - np.min(aug)) / (np.max(aug) - np.min(aug) + 1e-8)
X_aug.append(aug)
X_batch = np.array(X_aug, dtype=np.float32)
else:
If not augmenting, ensure normalisation
X_batch = np.array([
(x.squeeze() - np.min(x.squeeze())) / (np.max(x.squeeze()) - np.min(x.squeeze()) + 1e-8)
for x in X_batch
, dtype=np.float32])

Ensure correct shape for CNN
X_batch = np.expand_dims(X_batch, axis=-1)
return X_batch, y_batch

data_processing.py

@@ -75,15 +75,16 @@ def load_physionet_dataset(physionet_path, reference_file_path, X, y):
def preprocess_dataset(physionet_path, reference_file_path, X, y, fs=300):
X, y = load_physionet_dataset(physionet_path, reference_file_path, X, y)
X_filtered = np.array([butter_bandpass_filter(x, fs=fs) for x in X])
X_processed = normalise_dataset(X_filtered) # For DCNN, normalization is preferred
Do not normalise or augment here!
y_processed = y
X_balanced, y_balanced = smote_resampling(X_processed, y_processed)
X_balanced, y_balanced = smote_resampling(X_filtered, y_processed)
return X_balanced, y_balanced

def create_train_val_test_splits(fs=300):
X = []
y = []
- augmentor = ECG_Augmentor(fs=fs)
Use strong augmentations for training data
augmentor = ECG_Augmentor(fs=fs, strong=True)

Load and preprocess only, avoid unnecessary copies
X_processed, y_processed = preprocess_dataset(
"training2017/training2017",
@@ -94,7 +95,6 @@ def create_train_val_test_splits(fs=300):
X_processed = X_processed.astype(np.float32)
Reshape for CNN
X_cnn = np.expand_dims(X_processed, axis=-1)

Encode labels
le = LabelEncoder()
y_cnn = le.fit_transform(y_processed)
Split data
@@ -104,12 +104,20 @@ def create_train_val_test_splits(fs=300):
X_val, X_test, y_val, y_test = train_test_split(
X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
)

Augment only training data, process in batches to save RAM
batch_size = 500 # Adjust as needed for your RAM
X_train_aug = []
for i in range(0, len(X_train), batch_size):
batch = X_train[i:i+batch_size]
X_train_aug = [augmentor.augment(x.squeeze()) for x in batch]
For strong augmentation, optionally mixup with other signals in the batch
batch_aug = []
for x in batch:
if augmentor.strong and np.random.rand() < 0.2 and len(batch) > 1:
Mixup with another random signal in the batch
idx = np.random.choice([j for j in range(len(batch)) if not np.allclose(batch[j], x)])
x_aug = augmentor.augment(x.squeeze()), mix_signal=batch[idx].squeeze()
else:
x_aug = augmentor.augment(x.squeeze())
batch_aug.append(x_aug)
batch_aug = np.expand_dims(np.array(batch_aug, dtype=np.float32), axis=-1)
X_train_aug.append(batch_aug)
X_train_aug = np.concatenate(X_train_aug, axis=0)

model.py

@@ -10,9 +10,18 @@
from keras import regularizers
from focal_loss import SparseCategoricalFocalLoss
import numpy as np

+ from data_generator import ECGDataGenerator
+ from data_augmentor import ECG_Augmentor

X_train, y_train, X_val, y_val, X_test, y_test, le = create_train_val_test_splits()

+ train_augmentor = ECG_Augmentor(fs=300, strong=True)
+ train_gen = ECGDataGenerator(X_train, y_train, batch_size=64, augmentor=train_augmentor, shuffle=True)
+ val_gen = ECGDataGenerator(X_val, y_val, batch_size=64, augmentor=None, shuffle=False)
+
+ unique, counts = np.unique(y_train, return_counts=True)
+ print("Class distribution in y_train:", dict(zip(unique, counts)))
+
weights = compute_class_weight('balanced', classes=np.array([0,1,2]), y=y_train)
class_weights = dict(enumerate(weights))

@@ -73,19 +82,19 @@ def ecg_cnn(input_length, num_classes=3): # 0: afib, 1: normal, 2: other arrhyth
loss=SparseCategoricalFocalLoss(gamma=2),
metrics=['accuracy'])

- model.fit(X_train,
- y_train,
- epochs=100,
- batch_size=64,
- callbacks=callback,
- class_weight=class_weights,
- validation_data=(X_val, y_val)
-)
+ model.fit(
+ train_gen,
+ epochs=100,
+ callbacks=callback,
+ class_weight=class_weights,
+ validation_data=val_gen
+)

- test_loss, test_acc = model.evaluate(X_test, y_test)
+ test_gen = ECGDataGenerator(X_test, y_test, batch_size=64, augmentor=None, shuffle=False)
+ test_loss, test_acc = model.evaluate(test_gen)
print(f"Test accuracy: {test_acc:.4f}")

- y_pred = model.predict(X_test)
+ y_pred = model.predict(test_gen)
y_pred_classes = y_pred.argmax(axis=1)
print("Confusion Matrix:")

Comments 0

Please sign in to comment.